

# SpaceX Falcon 9 First Stage Landing Prediction

Data Science Capstone Project Report — Final Presentation

IBM Applied Data Science Capstone · Coursera

Author: Rahul Jha      Date: 26 July 2026

<https://github.com/pankh-ui/IBM-Applied-Data-Science-Capstone>

Contains all seven completed notebooks, the Plotly Dash Python file and this report

# Executive Summary

## Rubric section: Executive Summary

- Problem: SpaceX lists a Falcon 9 launch at \$62M against \$165M for its competitors, because it recovers and re-flies the first stage. Predicting whether that first stage lands is therefore the same as predicting what a launch will cost.
- Methodologies used: data collected from the SpaceX REST API and by scraping the Wikipedia Falcon 9 launch table; wrangled into a binary landing label; explored with Matplotlib/Seaborn charts and with SQL; presented through Folium maps and a Plotly Dash dashboard; then modelled with four tuned classifiers.
- Data: 90 Falcon 9 launches, 2010-2020, expanded to 80 features by one-hot encoding, split 72 training rows and 18 test rows.
- Key result 1 — the landing success rate rises from 0% in 2010-2013 to about 80% by 2019-2020. Experience, not payload, is the dominant factor.
- Key result 2 — KSC LC-39A is the strongest site at 76.9% success; CCAFS SLC-40 looks worst only because it flew the earliest missions.
- Key result 3 — payloads of 2 000-5 500 kg land 55% of the time, against 33% above 6 000 kg and 27% below 2 000 kg.
- Key result 4 — all four tuned models score 0.8333 on the test set. The Decision Tree leads on 10-fold cross-validation at 0.8625 and is the recommended model.
- Key result 5 — the best model's errors are all false positives (recall 1.00, precision 0.80), so any bid priced from it would be optimistic.

# Introduction

## Rubric section: Introduction

- Project background:

- SpaceX is the only launch provider that routinely recovers and re-flies the first stage of its rocket, and that recovery is what makes its advertised \$62M price possible when rivals charge upward of \$165M.
- When the first stage is lost, the booster cannot be re-flown and the real cost of that launch rises sharply. The landing outcome is the cost driver.
- Space Y, a new competitor, wants to bid against SpaceX and needs to estimate a SpaceX launch price before the launch happens.

- Problems we want to answer:

- Which factors determine whether the Falcon 9 first stage lands successfully?
- How do payload mass, launch site, orbit type and booster version interact with the landing outcome?
- Has the landing success rate improved over the ten years of the programme, and by how much?
- Which classification algorithm predicts a successful landing most reliably, and what kind of mistakes does it make?

- Deliverables: seven executed notebooks, an interactive Folium map, a Plotly Dash dashboard and this report — all published in the GitHub repository below.

# Results and Conclusion at a Glance

## Rubric section: Predictive Analysis Results · Conclusion

### What the data says

- Success rate by year: 0% (2010-13) → ~80% (2019-20).
- Best site: KSC LC-39A, 76.9% of its first stages recovered.
- Best payload band: 2 000-5 500 kg, 55% success.
- Perfect orbits: ES-L1, GEO, HEO, SSO (small samples). Worst busy orbit: GTO, ~50%.
- Booster generation matters: v1.0 never landed (0/5), v1.1 landed once (1/15), FT and B4 account for 22 of 24 successes.

### What the models say

- Four classifiers tuned with GridSearchCV, 10-fold CV.
- Validation accuracy: Decision Tree 0.8625, SVM 0.8482, KNN 0.8482, Logistic Regression 0.8464.
- Test accuracy: 0.8333 for all four (15 of 18 launches).
- Recommended model: Decision Tree — best cross-validated score and the most interpretable for a costing team.
- Error profile: 12 true positives, 3 true negatives, 3 false positives, 0 false negatives.

### Conclusion

A launch's cost can be estimated before it flies. The strongest predictor is flight number — a proxy for accumulated engineering experience — followed by booster generation, launch site and orbit. A Decision Tree on those features recovers the landing outcome for 15 of 18 unseen launches, which is accurate enough for Space Y to price a competing bid, provided the optimistic bias from its three false positives is corrected for.

# Project Repository and Contents

Rubric section: GitHub Repository URL

<https://github.com/pankh-ui/IBM-Applied-Data-Science-Capstone>

## Completed notebooks and Python files in the repository

- 1-jupyter-labs-spacex-data-collection-api.ipynb
- 2-jupyter-labs-webscraping.ipynb
- 3-labs-jupyter-spacex-Data-wrangling.ipynb
- 4-jupyter-labs-eda-dataviz.ipynb
- 5-jupyter-labs-eda-sql-coursera\_sqlite.ipynb
- 6-lab-jupyter-launch-site-location.ipynb
- 7-SpaceX-Machine-Learning-Prediction.ipynb
- spacex\_dash\_app.py (Plotly Dash application)
- Data\_Science\_Capstone\_Project\_Report.pdf (this file)

## Where each required section appears in this report

- Executive Summary — slide 2
- Introduction — slide 3
- Results and Conclusion at a glance — slide 4
- Data Collection and Data Wrangling Methodology — slides 6-7
- EDA and Data Visualization Methodology — slide 8
- EDA with SQL and Interactive Analytics Methodology — slides 9-10
- Predictive Analysis (Classification) Methodology — slide 11
- EDA with Visualization Results — slides 12-15
- EDA with SQL Results and Queries — slides 16-20
- Folium Map Results — slides 21-22
- Plotly Dash Dashboard Results — slides 23-24
- Predictive Analysis Results — slides 25-28
- Conclusion and Innovative Insights — slides 29-30
- Final summary of the submission — slide 31

# Data Collection Methodology — API and Web Scraping

## Rubric section: Data Collection and Data Wrangling Methodology

### SpaceX REST API

- Flow: GET `api.spacexdata.com/v4/launches/past` → `response.json()` → `pd.json_normalize()` → ID lookups → filter Falcon 9 → impute nulls → `dataset_part_1.csv`
- `requests.get(spacex_url)` retrieves every past launch as nested JSON.
- `pd.json_normalize()` flattens it into a dataframe.
- `getBoosterVersion`, `getLaunchSite`, `getPayloadData` and `getCoreData` call the rockets, launchpads, payloads and cores endpoints to turn IDs into real values.
- Rows with multiple cores or payloads are dropped (those are Falcon Heavy).
- Filter: `data['BoosterVersion'] != 'Falcon 1'` leaves 90 Falcon 9 launches.

### Wikipedia web scraping

- Flow: `requests.get(static_url)` → `BeautifulSoup(html, 'html.parser')` → `find_all('table')` → parse headers → row-by-row extraction → `spacex_web_scraped.csv`
- `static_url` is the 9 June 2021 snapshot of 'List of Falcon 9 and Falcon Heavy launches', so the scrape is reproducible.
- `soup.title` returns 'List of Falcon 9 and Falcon Heavy launches - Wikipedia', confirming the right page was fetched.
- The third wikipable holds the launch records; `extract_column_from_header()` builds the column names.
- Helper functions `date_time()`, `booster_version()`, `landing_status()` and `get_mass()` clean each cell before it enters `launch_dict`.

# Data Wrangling Methodology

## Rubric section: Data Collection and Data Wrangling Methodology

- Missing values: PayloadMass nulls replaced by the column mean; LandingPad keeps its 26 nulls because 'no landing pad' is itself information.
- Exploratory counts: launches per site (CCAFS SLC 40 = 55, KSC LC 39A = 22, VAFB SLC 4E = 13), occurrences of each orbit, and mission outcome per orbit.
- The Outcome column holds eight values that mix the result with the landing place: True ASDS, True RTLS, True Ocean, False ASDS, False RTLS, False Ocean, None ASDS and None None.
- `bad_outcomes = {False ASDS, False Ocean, False RTLS, None ASDS, None None}`. Anything in that set becomes Class = 0; everything else becomes Class = 1.
- That single transformation turns an unusable text column into the binary training label the whole project depends on.
- Result: `df['Class'].mean() = 0.667` — a two-thirds success rate across 90 launches.
- Feature engineering: Orbit, LaunchSite, LandingPad and Serial one-hot encoded with `pd.get_dummies`, giving an 80-column float64 matrix saved as `dataset_part_3.csv`.

# EDA and Data Visualization Methodology

## Rubric section: EDA and Data Visualization Methodology

- Scatter — Flight Number vs Payload Mass: tests whether success depends on the weight carried or on how far into the programme the launch happened.
- Scatter — Flight Number vs Launch Site: separates a genuine site effect from the fact that different pads were busy in different eras.
- Scatter — Payload Mass vs Launch Site: shows which pads handle which mission classes.
- Bar — success rate by orbit type: a rate comparison across categories, so a bar chart is the correct form.
- Scatter — Flight Number vs Orbit and Payload Mass vs Orbit: distinguishes an orbit effect from an experience or payload effect.
- Line — yearly average success rate: a time series, so a line chart with markers.
- Colour encodes the target class throughout (Class = 1 landed, Class = 0 did not), so every chart doubles as a visual check on class separability.
- Feature engineering follows the charts: the variables that separate the classes visually are the ones one-hot encoded for the models.



# EDA with SQL — Methodology

## Rubric section: EDA with SQL Slides/Queries

- The launch table is loaded into SQLite with `df.to_sql('SPACEXTBL', con)` and queried through the `%sql` magic in Jupyter.
- A clean table is created first: `CREATE TABLE SPACEXTABLE AS SELECT * FROM SPACEXTBL WHERE Date IS NOT NULL.`
- Ten queries answer ten specific business questions:
  - unique launch sites; five records beginning with 'CCA'; total NASA (CRS) payload; average F9 v1.1 payload; date of the first ground-pad landing
  - drone-ship successes carrying 4 000-6 000 kg; counts of successful and failed mission outcomes; boosters carrying the maximum payload, via a subquery
  - 2015 drone-ship failures with the month name extracted by `substr()` and `CASE`; landing outcomes ranked by count between 2010-06-04 and 2017-03-20
- The techniques exercised are `DISTINCT`, `LIKE` with a wildcard, `SUM`/`AVG`/`MIN` aggregates, `BETWEEN`, `GROUP BY`, `ORDER BY`, a scalar subquery and a `CASE` expression.
- Every query and its returned rows appear on the results slides that follow.

# Interactive Visual Analytics — Methodology

## Rubric section: Folium Map · Plotly Dash Methodology

### Folium map objects

- folium.Circle plus folium.Marker with a DivIcon label at each launch site, so the geographic spread is visible at a glance.
- MarkerCluster groups the launch records at each pad; markers are green for a successful landing and red for a failure. Without clustering the records would overlap exactly, because a pad has one coordinate.
- folium.PolyLine plus a haversine distance function measures how far a pad sits from the coastline, a railway, a highway and the nearest city.
- MousePosition adds a live latitude/longitude readout for picking those points.

### Plotly Dash components

- dcc.Dropdown(id='site-dropdown') offers All Sites plus each individual launch site.
- A pie-chart callback returns either the share of all successful launches by site, or the success-versus-failure split for one selected site.
- dcc.RangeSlider(id='payload-slider') spans 0-10 000 kg in 1 000 kg steps.
- A scatter callback takes both the dropdown and the slider as Inputs and plots payload against landing outcome, coloured by booster version category.
- Two Outputs driven from one dropdown demonstrate the reactive callback pattern the lab requires.

# Predictive Analysis (Classification) — Methodology

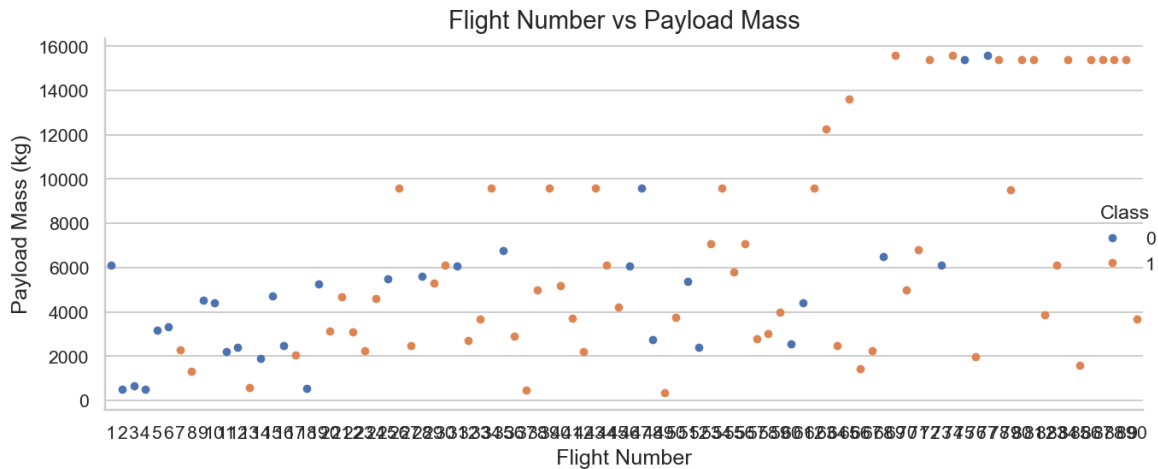
## Rubric section: Predictive Analysis (Classification)

- Target: `Y = df['Class'].to_numpy()` — 1 if the first stage landed, 0 if it did not.
- Features: the 80-column one-hot encoded matrix, standardized with `StandardScaler().fit_transform(X)` so that no feature dominates by scale.
- Split: `train_test_split(X, Y, test_size=0.2, random_state=2)` → 72 training rows and 18 test rows.
- Four algorithms, each tuned by `GridSearchCV` with 10-fold cross-validation:
  - Logistic Regression — `C`, `penalty`, `solver`
  - Support Vector Machine — `kernel` (linear, rbf, poly, sigmoid), `C`, `gamma`
  - Decision Tree — `criterion`, `splitter`, `max_depth`, `max_features`, `min_samples_leaf`, `min_samples_split`
  - K-Nearest Neighbours — `n_neighbors`, `algorithm`, `p`
- Evaluation: `best_score_` gives the cross-validated accuracy, `.score(X_test, Y_test)` the held-out accuracy, and a confusion matrix the error profile.
- The test set is scored once, after tuning, so the reported figure is not optimistic.

# Flight Number vs Payload Mass and Flight Number vs Launch Site

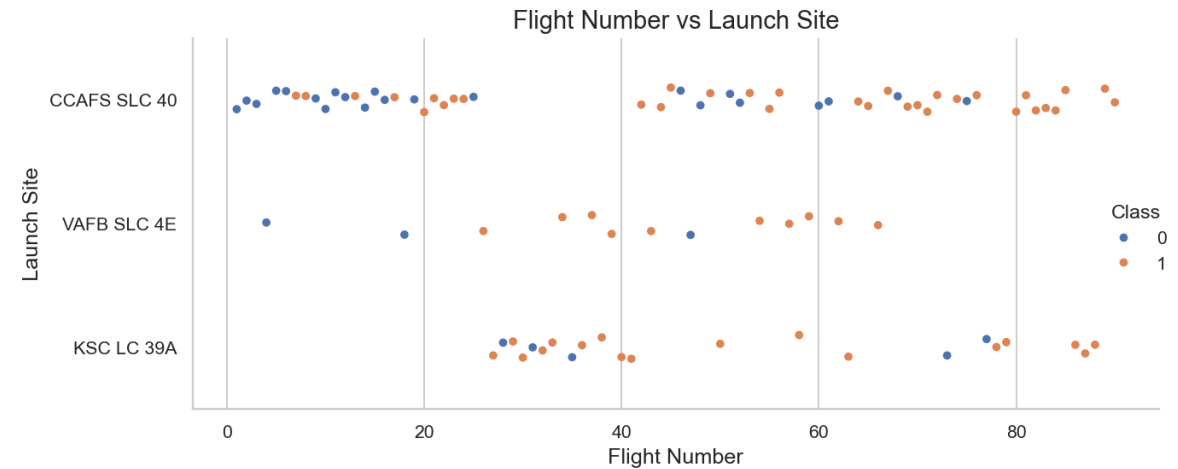
## Rubric section: EDA with Visualization

### Flight Number vs Payload Mass



- Successes cluster on the right: above flight 60 almost every launch lands, whatever the payload.
- Payloads over 10 000 kg are rare and still land, so weight alone does not doom a landing.
- FlightNumber is therefore a strong candidate feature.

### Flight Number vs Launch Site

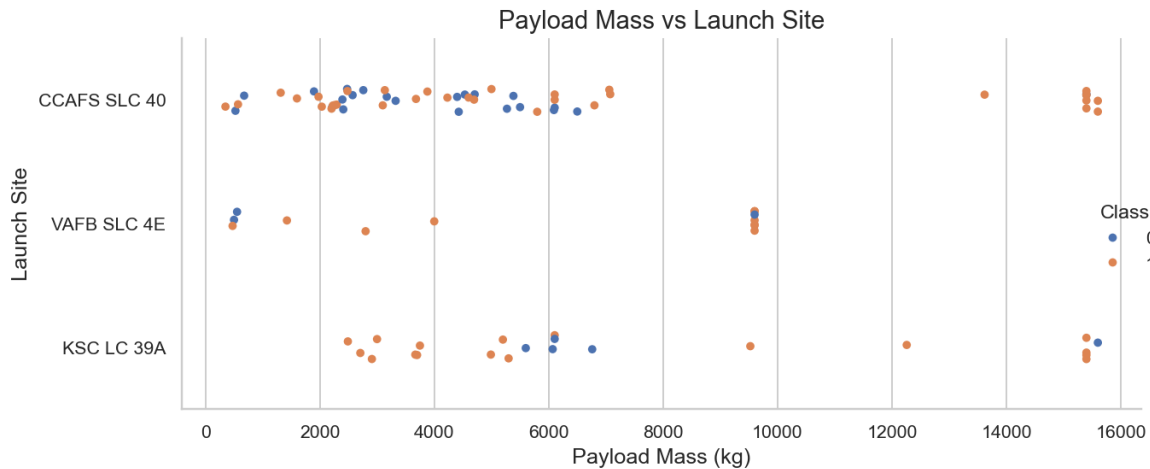


- CCAFS SLC 40 flew almost all of the first 25 missions, where the failures sit.
- KSC LC 39A appears only after flight 25 and is almost entirely successful.
- Site quality and programme era are confounded — both belong in the model.

# Payload Mass vs Launch Site and Success Rate by Orbit Type

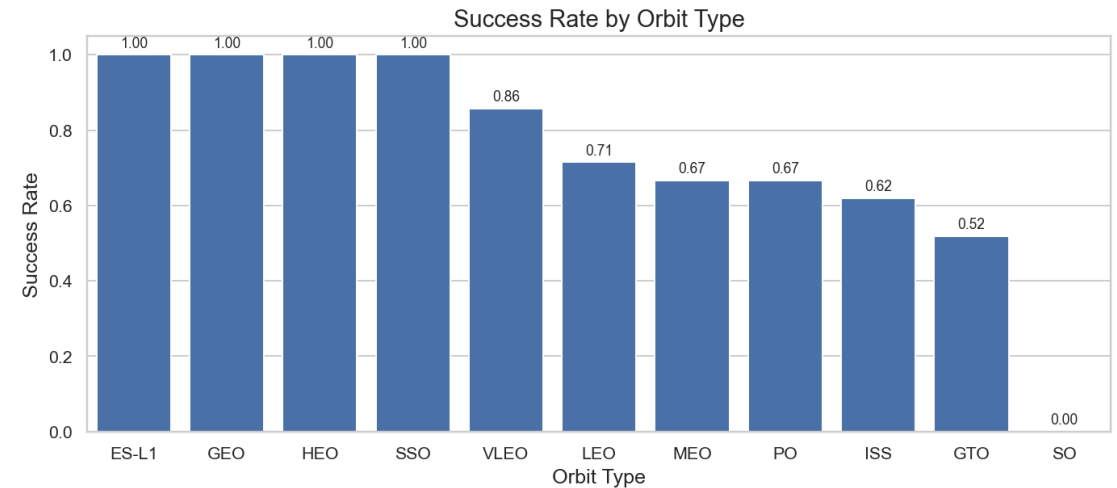
## Rubric section: EDA with Visualization

### Payload Mass vs Launch Site



- VAFB SLC 4E has no launch above 10 000 kg; the heaviest missions all fly from Florida.
- At the Florida pads, launches above roughly 7 000 kg succeed more often than the mid-weight range.
- Launch site and payload mass interact rather than acting independently.

### Success Rate by Orbit Type

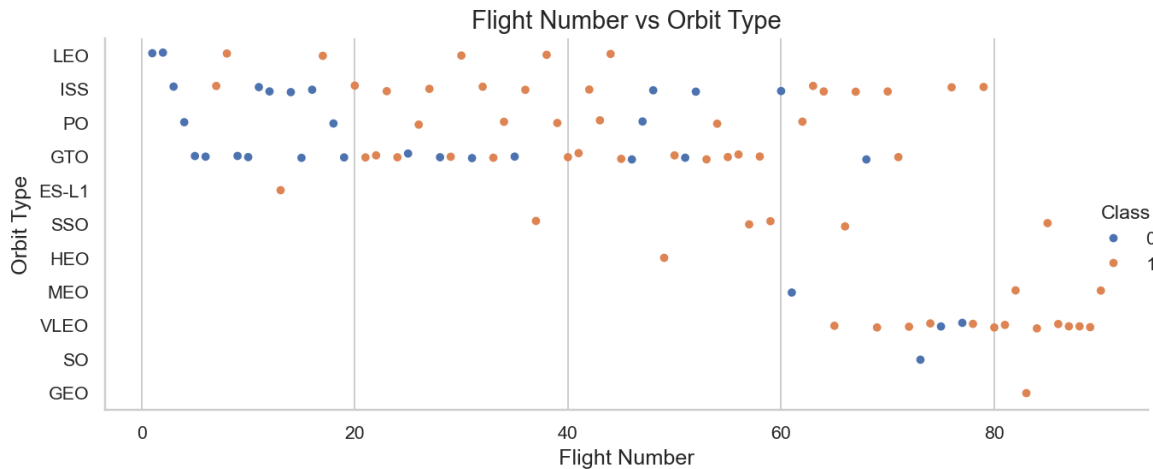


- ES-L1, GEO, HEO and SSO record 100% success; SO records 0%.
- Each of those orbits has only a handful of launches, so the rate is not yet statistically meaningful.
- GTO, the busiest orbit, sits near 0.5 — transfer missions leave the least fuel for the landing burn.

# Flight Number vs Orbit Type and Payload Mass vs Orbit Type

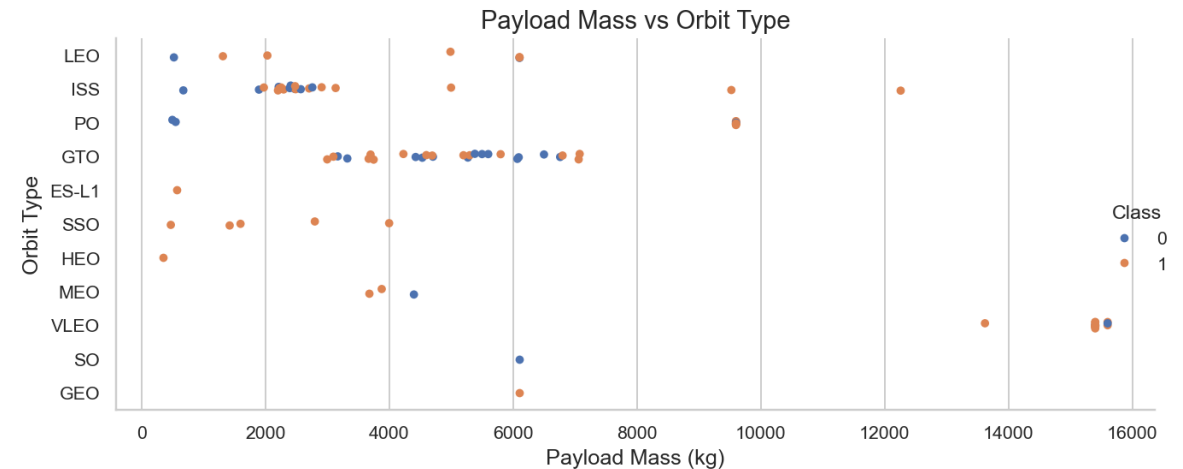
## Rubric section: EDA with Visualization

### Flight Number vs Orbit Type



- In LEO, success rises clearly with flight number — experience matters.
- In GTO there is no such pattern; outcomes stay mixed across the whole range.
- The 100% orbits appear as isolated points, reinforcing the sample-size caveat.

### Payload Mass vs Orbit Type



- Heavy payloads land successfully in LEO, ISS and PO.
- For GTO the outcome is mixed at every payload band.
- It is the orbit-by-payload interaction, not payload alone, that carries signal.

# Launch Success Yearly Trend

## Rubric section: EDA with Visualization



- The mean landing success rate per year, 2010 to 2020.
- The rate is 0% until 2013, then climbs almost monotonically to roughly 80% by 2019-2020.
- The dip in 2018 interrupts the trend without reversing it.
- This is the single strongest signal in the dataset, and it is temporal rather than physical: SpaceX solved the landing problem by iterating, not by flying easier missions.
- Consequence for the model: any feature correlated with time — flight number, booster serial, block number — will carry predictive weight.

# EDA with SQL — Task 1 · Task 2

## Rubric section: EDA with SQL Slides/Queries

### Task 1 — Unique launch sites

```
SELECT DISTINCT Launch_Site FROM SPACEXTABLE;
```

#### Task 1 — Unique launch sites

| Launch_Site  |
|--------------|
| CCAFS LC-40  |
| VAFB SLC-4E  |
| KSC LC-39A   |
| CCAFS SLC-40 |

- Result: CCAFS LC-40, CCAFS SLC-40, KSC LC-39A, VAFB SLC-4E.
- Three Florida pads and one in California — the two clusters seen on the Folium map.

### Task 2 — Sites beginning with 'CCA'

```
SELECT Date, Launch_Site, Booster_Version, Payload, PAYLOAD_MASS_KG_ FROM SPACEXTABLE WHERE Launch_Site LIKE 'CCA%' LIMIT 5;
```

#### Task 2 — 5 records where launch site begins with 'CCA'

| Date       | Launch_Site | Booster_Version | Payload                                     | PAYLOAD_MASS_KG_ |
|------------|-------------|-----------------|---------------------------------------------|------------------|
| 2010-06-04 | CCAFS LC-40 | F9 v1.0 B0003   | Dragon Spacecraft Qualification flight      | 0                |
| 2010-12-08 | CCAFS LC-40 | F9 v1.0 B0004   | Dragon B0004 flight C1, two CubeSats, barre | 0                |
| 2012-05-22 | CCAFS LC-40 | F9 v1.0 B0005   | Dragon demo flight C2                       | 525              |
| 2012-10-08 | CCAFS LC-40 | F9 v1.0 B0006   | SpaceX CRS-1                                | 500              |
| 2013-03-01 | CCAFS LC-40 | F9 v1.0 B0007   | SpaceX CRS-2                                | 677              |

- The LIKE wildcard matches both Cape Canaveral pads.
- Result: the five earliest missions, all F9 v1.0 with payloads of 0-677 kg — the Dragon demonstration flights.



# EDA with SQL — Task 3 · Task 4

## Rubric section: EDA with SQL Slides/Queries

### Task 3 — Total NASA (CRS) payload

```
SELECT SUM(PAYLOAD_MASS__KG_) AS Total_Payload_Mass
FROM SPACEXTABLE WHERE Customer = 'NASA (CRS)';
```

#### Task 3 — Total payload mass carried by boosters from NASA (CRS)

| Total_Payload_Mass_KG |
|-----------------------|
| 45596                 |

- Result: 45 596 kg.
- CRS is the largest single customer block, which is why ISS-bound missions dominate the LEO and ISS orbit categories.

### Task 4 — Average F9 v1.1 payload

```
SELECT AVG(PAYLOAD_MASS__KG_) AS Avg_Payload_Mass
FROM SPACEXTABLE WHERE Booster_Version = 'F9 v1.1';
```

#### Task 4 — Average payload mass carried by booster version F9 v1.1

| Avg_Payload_Mass_KG |
|---------------------|
| 2928.4              |

- Result: 2 928.4 kg.
- Well below the fleet average — v1.1 was the early, less capable variant, and it also has the worst landing record (1 success in 15).

# EDA with SQL — Task 5 · Task 6

## Rubric section: EDA with SQL Slides/Queries

### Task 5 — First ground-pad landing

```
SELECT MIN(Date) AS First_Successful_Ground_Landing FROM SPACEXTABLE
WHERE Landing_Outcome = 'Success (ground pad)';
```

#### Task 5 — Date of the first successful landing outcome on ground pad

| First_Successful_Ground_Landing |
|---------------------------------|
| 2015-12-22                      |

- Result: 2015-12-22.
- This date is the inflection point of the yearly success curve.

### Task 6 — Drone-ship success, 4 000-6 000 kg

```
SELECT DISTINCT Booster_Version FROM SPACEXTABLE
WHERE Landing_Outcome = 'Success (drone ship)'
AND PAYLOAD_MASS__KG_ > 4000 AND PAYLOAD_MASS__KG_ < 6000;
```

#### Task 6 — Boosters: success on drone ship, payload 4000-6000 kg

| Booster_Version | PAYLOAD_MASS__KG_ |
|-----------------|-------------------|
| F9 FT B1022     | 4696              |
| F9 FT B1026     | 4600              |
| F9 FT B1021.2   | 5300              |
| F9 FT B1031.2   | 5200              |

- Result: F9 FT B1022, B1026, B1021.2 and B1031.2.
- All Full Thrust cores, two of them re-flown (.2) — re-used boosters land as reliably as new ones.

# EDA with SQL — Task 7 · Task 8

## Rubric section: EDA with SQL Slides/Queries

### Task 7 — Mission outcome counts

```
SELECT Mission_Outcome, COUNT(*) AS Total
FROM SPACEXTABLE GROUP BY Mission_Outcome;
```

#### Task 7 — Total number of successful and failed mission outcomes

| Mission_Outcome                  | Total |
|----------------------------------|-------|
| Failure (in flight)              | 1     |
| Success                          | 98    |
| Success                          | 1     |
| Success (payload status unclear) | 1     |

- Result: 98 'Success', 1 'Success' with a trailing space, 1 'Success (payload status unclear)' and 1 'Failure (in flight)'.
- Mission success and landing success are different problems: the payload almost always reaches orbit even when the booster is lost.

### Task 8 — Boosters at maximum payload

```
SELECT DISTINCT Booster_Version FROM SPACEXTABLE WHERE PAYLOAD_MASS_KG_ =
(SELECT MAX(PAYLOAD_MASS_KG_) FROM SPACEXTABLE);
```

Task 8 — Booster versions that carried the maximum payload mass

| Booster_Version | PAYLOAD_MASS_KG_ |
|-----------------|------------------|
| F9 B5 B1048.4   | 15600            |
| F9 B5 B1049.4   | 15600            |
| F9 B5 B1051.3   | 15600            |
| F9 B5 B1056.4   | 15600            |
| F9 B5 B1048.5   | 15600            |
| F9 B5 B1051.4   | 15600            |
| F9 B5 B1049.5   | 15600            |
| F9 B5 B1060.2   | 15600            |
| F9 B5 B1058.3   | 15600            |
| F9 B5 B1051.6   | 15600            |
| F9 B5 B1060.3   | 15600            |
| F9 B5 B1049.7   | 15600            |

- A scalar subquery finds the maximum, the outer query names the boosters.
- Result: twelve boosters, every one a B5 core — only the final Block 5 design lifts the heaviest payloads.

# EDA with SQL — Task 9 · Task 10

## Rubric section: EDA with SQL Slides/Queries

### Task 9 — 2015 drone-ship failures

```
SELECT CASE substr(Date,6,2) WHEN '01' THEN 'January' ... END AS Month,  
       Landing_Outcome, Booster_Version, Launch_Site FROM SPACEXTABLE  
WHERE substr(Date,1,4)='2015' AND Landing_Outcome='Failure (drone ship)';
```

### Task 9 — 2015 failure landing outcomes on drone ship (month, booster, site)

| Month   | Landing_Outcome      | Booster_Version | Launch_Site |
|---------|----------------------|-----------------|-------------|
| January | Failure (drone ship) | F9 v1.1 B1012   | CCAFS LC-40 |
| April   | Failure (drone ship) | F9 v1.1 B1015   | CCAFS LC-40 |

- substr() extracts the month, CASE converts it to a name.
- Result: January and April 2015, both F9 v1.1 from CCAFS LC-40 — the attempts that preceded the first success.

### Task 10 — Landing outcomes ranked

```
SELECT Landing_Outcome, COUNT(*) AS Outcome_Count FROM SPACEXTABLE  
WHERE Date BETWEEN '2010-06-04' AND '2017-03-20'  
GROUP BY Landing_Outcome ORDER BY Outcome_Count DESC;
```

Task 10 — Landing outcomes ranked, 2010-06-04 to 2017-03-20

| Landing_Outcome        | Outcome_Count |
|------------------------|---------------|
| No attempt             | 10            |
| Success (drone ship)   | 5             |
| Failure (drone ship)   | 5             |
| Success (ground pad)   | 3             |
| Controlled (ocean)     | 3             |
| Uncontrolled (ocean)   | 2             |
| Failure (parachute)    | 2             |
| Precluded (drone ship) | 1             |

- Result: No attempt 10, drone-ship success 5, drone-ship failure 5, ground-pad success 3, controlled ocean 3.
- For most of the first seven years the commonest outcome was not even trying to land.

# Folium Map — All Launch Sites Marked on the Map · Launch Outcomes Clustered by Site

## Rubric section: Folium Map

### All Launch Sites Marked on the Map



- Objects: folium.Circle plus a folium.Marker with a DivIcon text label at each of the four sites, on a map centred on NASA Johnson Space Center.
- Every site is coastal — three on the Atlantic in Florida, one on the Pacific in California — so a failed launch falls over water.
- The Florida pads sit at about  $28.5^{\circ}$  N against Vandenberg's  $34.6^{\circ}$  N, gaining more free velocity from the Earth's rotation.
- No site is inland, which sharply constrains where a competitor could build.

### Launch Outcomes Clustered by Site

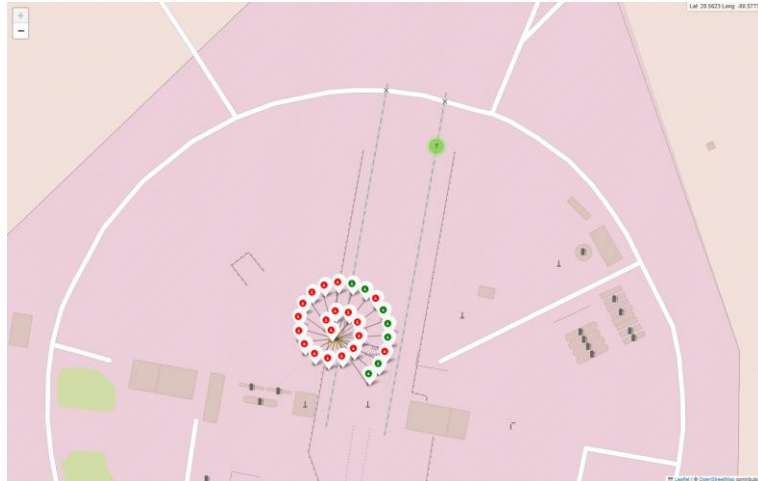


- Objects: a MarkerCluster holding all 56 launch records, each coloured green for a successful landing and red for a failure.
- Cluster counts make the launch volume obvious — the Florida cluster dwarfs Vandenberg.
- Clustering is what makes the map readable: all records at one pad share a single coordinate and would otherwise overlap exactly.

# Folium Map — Markers Expanded at CCAFS SLC-40 · Launch Site Proximities — CCAFS SLC-40

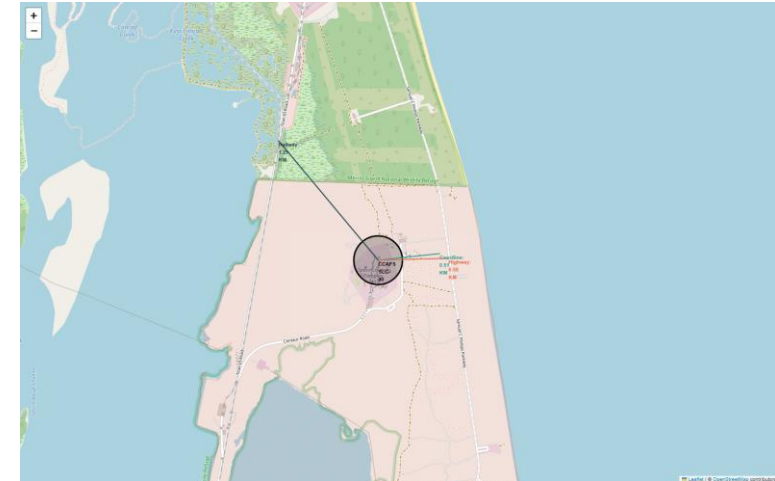
## Rubric section: Folium Map

### Markers Expanded at CCAFS SLC-40



- The cluster is expanded so the individual green and red markers are visible.
- Red markers clearly outnumber green at this pad, matching the EDA finding that it flew the early, failure-prone missions.
- The green markers are the later flights, so the pad improves over time rather than being uniformly poor.

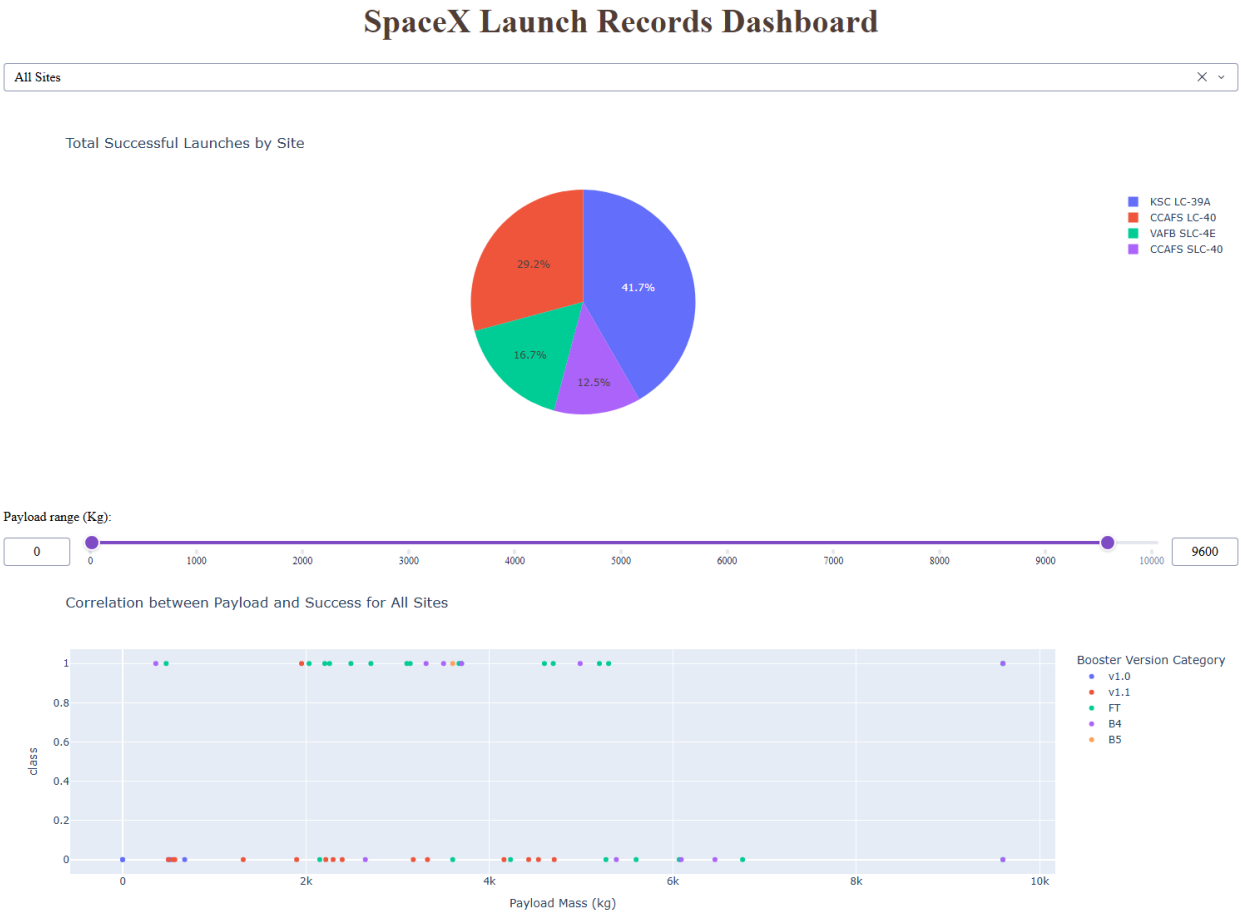
### Launch Site Proximities — CCAFS SLC-40



- Objects: folium.PolyLine from the pad to each point of interest, labelled with a haversine distance computed in Python.
- Coastline 0.51 km, highway 0.58 km, railway 1.29 km, nearest city (Titusville) 23.19 km.
- Pads sit beside transport links, which are needed to move stages and propellant.
- They stay over 20 km from population centres — the safety constraint that dictates site selection.

# Plotly Dash Dashboard — All Sites Selected

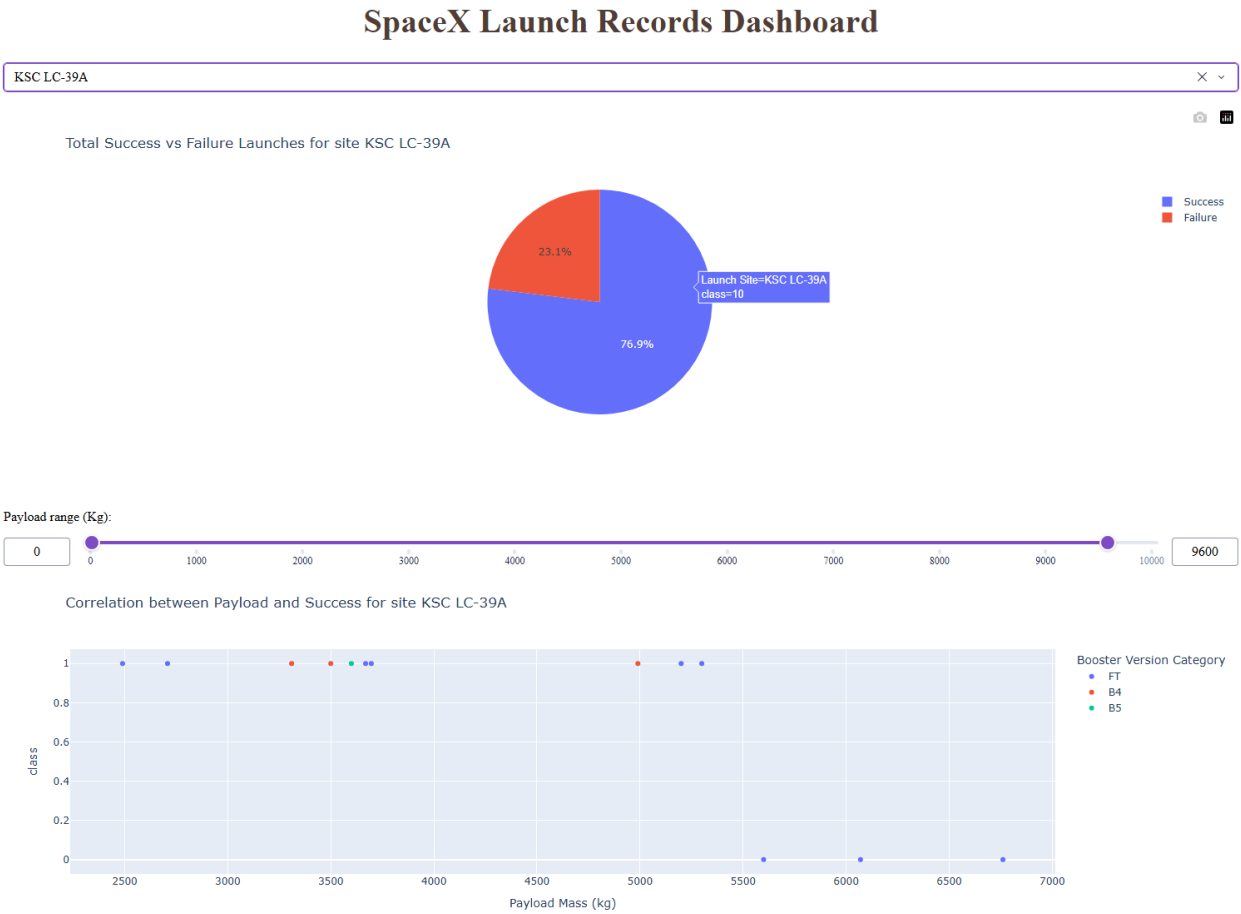
## Rubric section: Plotly Dash Dashboard



- Components on screen: dcc.Dropdown set to All Sites, the success pie chart, the 0-10 000 kg dcc.RangeSlider and the payload-versus-outcome scatter chart.
- Pie result: KSC LC-39A contributes 41.7% of all successful launches, CCAFS LC-40 29.2%, VAFB SLC-4E 16.7% and CCAFS SLC-40 12.5%.
- Scatter: the 2 000-5 500 kg band lands 55% of the time (18 of 33), against 33% (3 of 9) above 6 000 kg and 27% (3 of 11) below 2 000 kg.
- By booster version, FT and B4 cores account for 22 of the 24 successes; v1.0 never lands (0 of 5) and v1.1 lands once in 15.
- Reading: the dashboard turns the static EDA into something a bidding team can interrogate site by site.

# Plotly Dash Dashboard — KSC LC-39A Selected

## Rubric section: Plotly Dash Dashboard

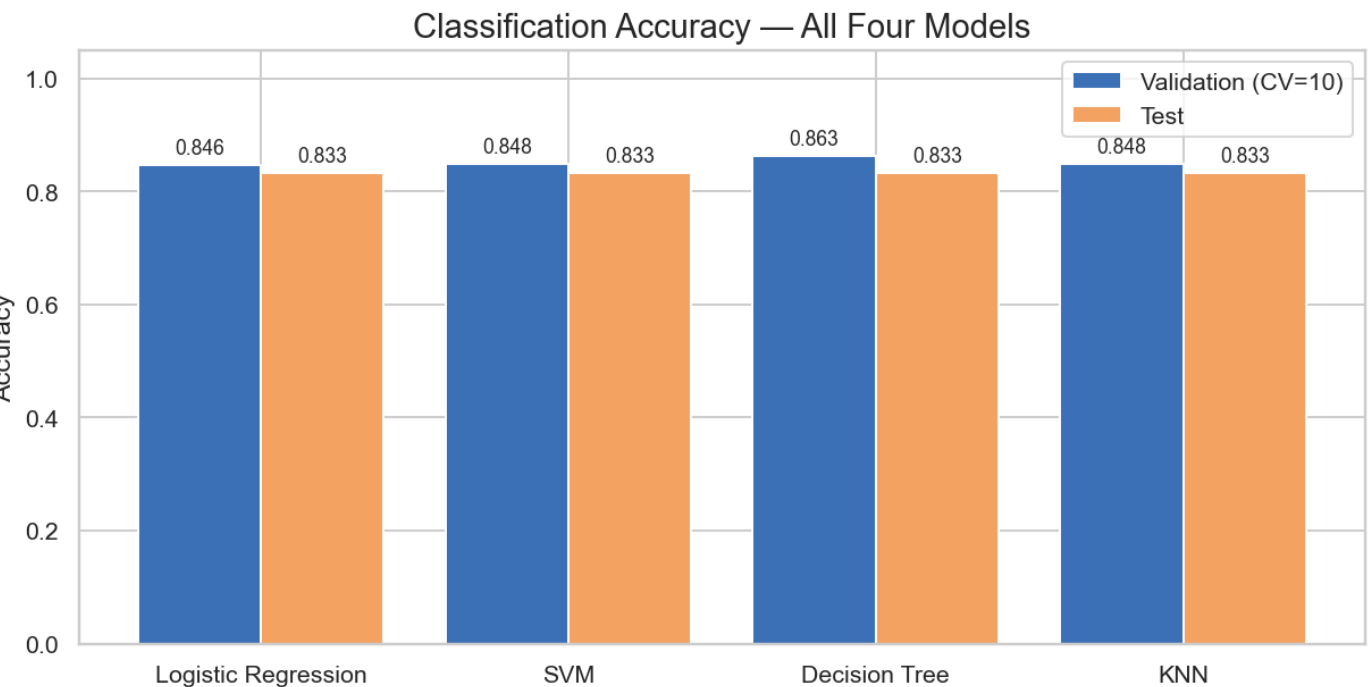


- Selecting one site fires both callbacks: the pie switches to that site's success-versus-failure split and the scatter filters to its launches.
- Pie result: KSC LC-39A recovers 76.9% of its first stages — the best ratio of any site in the dataset.
- Scatter: its successes concentrate in the lighter payload range and are flown almost entirely by FT and B4 boosters.
- One dropdown driving two independent Output components is exactly the reactive callback behaviour the lab asks for.



# Predictive Analysis Results — Classification Accuracy

## Rubric section: Predictive Analysis Results



- Validation accuracy, 10-fold cross-validation on the 72 training rows: Decision Tree 0.8625, SVM 0.8482, KNN 0.8482, Logistic Regression 0.8464.
- Test accuracy on the held-out 18 launches: 0.8333 for all four models — 15 correct out of 18.
- The tie is a property of the test set, not the models: with 18 rows accuracy can only move in steps of 5.6 percentage points.
- The cross-validated score is the fairer ranking, and the Decision Tree leads it by about 1.4 points.

# Predictive Analysis Results — Tuned Hyperparameters

## Rubric section: Predictive Analysis Results

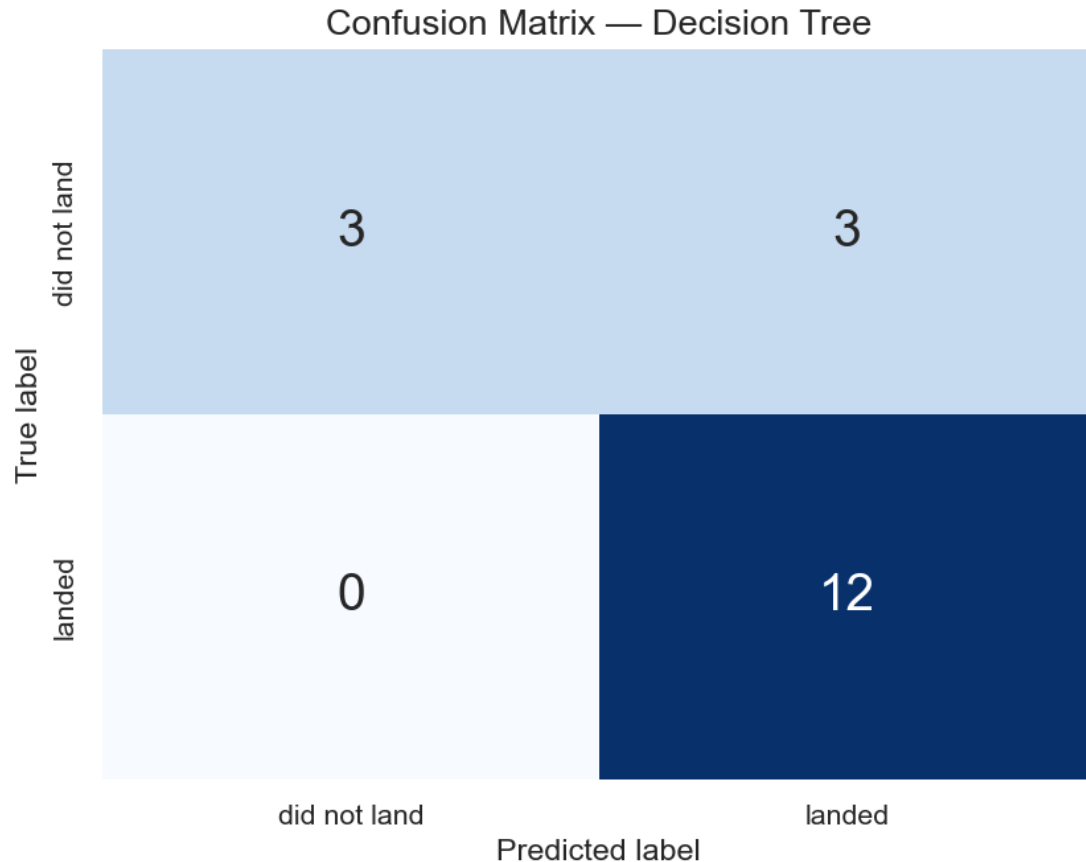
### Model comparison — validation vs test accuracy

| Model               | Best validation accuracy (CV=10) | Test accuracy |
|---------------------|----------------------------------|---------------|
| Logistic Regression | 0.8464                           | 0.8333        |
| SVM                 | 0.8482                           | 0.8333        |
| Decision Tree       | 0.8625                           | 0.8333        |
| KNN                 | 0.8482                           | 0.8333        |

- Logistic Regression: validation 0.8464, test 0.8333 — {'C': 0.01, 'penalty': 'l2', 'solver': 'lbfgs'}
- SVM: validation 0.8482, test 0.8333 — {'C': 1.0, 'gamma': 0.03162277660168379, 'kernel': 'sigmoid'}
- Decision Tree: validation 0.8625, test 0.8333 — {'criterion': 'gini', 'max\_depth': 6, 'max\_features': 'sqrt', 'min\_samples\_split': 2}
- KNN: validation 0.8482, test 0.8333 — {'algorithm': 'auto', 'n\_neighbors': 10, 'p': 1}
- Each model was tuned by GridSearchCV over the grid listed in the methodology, refitted on the full training set, then scored once on the untouched test set.

# Predictive Analysis Results — Confusion Matrix (Decision Tree)

## Rubric section: Predictive Analysis Results



- Counts on the 18 test launches: 12 true positives, 3 true negatives, 3 false positives, 0 false negatives.
- Recall on the landed class is 1.00 — the model never misses a successful landing.
- Precision on that class is  $12/15 = 0.80$ .
- Every error is the same kind: a booster the model expects to land that in fact does not. The model has no way of being pessimistic.
- Business consequence: a bid priced from this model would systematically underestimate cost. Raising the decision threshold, or weighting the loss against false positives, is the next iteration.

# Predictive Analysis Results — Best Performing Model

## Rubric section: Predictive Analysis Results

- Best model: Decision Tree. Validation accuracy 0.8625, test accuracy 0.8333.
- Why it wins: highest cross-validated score across 10 folds, and the tie on the test set means the cross-validated figure is the only one that discriminates.
- Why it is the right choice regardless: a tree can be printed as a set of rules, so a bidding team can see why a launch was priced the way it was. Neither SVM with a sigmoid kernel nor KNN offers that.
- Tuned configuration: entropy criterion, best splitter, limited depth with min\_samples\_leaf and min\_samples\_split constraints — a deliberately shallow tree, which suits 72 training rows.
- Caveat: 90 launches and 80 features is a very wide, very shallow problem. More launches, or feature selection down to the handful of variables that matter, would firm up every figure on this slide.

# Conclusion

## Rubric section: Conclusion

- Experience is the dominant factor. The yearly success rate climbs from 0% in 2010-2013 to roughly 80% by 2019-2020, and flight number separates the classes better than any physical variable.
- Launch site matters, but partly as a proxy for era. KSC LC-39A recovers 76.9% of its boosters; CCAFS SLC 40 looks poor mainly because it flew the early missions.
- Orbit matters. ES-L1, GEO, HEO and SSO show 100% success on small samples, while GTO — the busiest orbit — sits near 50%.
- Payload matters least of the four. The 2 000-5 500 kg band lands 55% of the time against 33% above 6 000 kg, but heavy payloads to LEO, ISS and PO still land.
- Booster generation is a clean predictor: v1.0 never landed, v1.1 landed once in 15, and the FT and B4 generations account for 22 of 24 successes.
- The Decision Tree, tuned by 10-fold cross-validation, predicts the landing outcome for 15 of 18 unseen launches (83.33%), with a cross-validated accuracy of 86.25%.
- Business answer: Space Y can estimate a SpaceX launch price before the launch flies, from site, orbit, payload and booster version — provided it corrects for the model's three false positives, which all bias the estimate optimistically.

# Innovative Insights

## Rubric section: Innovative Insights and Creativity

- The strongest predictor is temporal, not physical. Flight number — a proxy for accumulated engineering experience — separates landed from lost better than payload mass does. SpaceX solved this problem by iterating, and any competitor should expect to pay the same learning cost.
- Perfect orbit success rates are an artefact of volume. ES-L1, GEO, HEO and SSO sit at 100% on a handful of launches each, while GTO looks weak largely because it has the most attempts. Any orbit-based pricing rule needs a sample-size caveat attached.
- Re-flown boosters are not riskier. Two of the four boosters that landed on a drone ship carrying 4 000-6 000 kg were re-used cores, and the whole FT/B4 generation accounts for 22 of 24 successes — re-use is not the risk factor intuition suggests.
- The model's errors are one-sided. Recall is 1.00 and precision 0.80: every mistake is an over-optimistic prediction. A cost model with that asymmetry will always under-price, which is a business risk that raw accuracy hides completely.
- Site geography is a constraint, not a coincidence. Every pad is within 0.6 km of a highway and 1.3 km of a railway, yet over 23 km from the nearest town — a pattern a new entrant would have to reproduce.

# Final Summary of the Submission

Rubric section: All Required Sections · Repository URL

<https://github.com/pankh-ui/IBM-Applied-Data-Science-Capstone>

## This PDF contains every required section

- Executive Summary (slide 2) and Introduction (slide 3)
- Data Collection and Data Wrangling Methodology (slides 6-7)
- EDA and Data Visualization Methodology (slide 8)
- EDA with SQL and Interactive Analytics Methodology (slides 9-10)
- Predictive Analysis (Classification) Methodology (slide 11)
- EDA with Visualization results — 7 charts (slides 12-15)
- EDA with SQL — all 10 queries and their results (slides 16-20)
- Folium Map results — 4 maps (slides 21-22)
- Plotly Dash Dashboard results — 2 screenshots (slides 23-24)
- Predictive Analysis Results (slides 25-28)
- Conclusion and Innovative Insights (slides 29-30)

## The repository contains every notebook and Python file

- Seven executed Jupyter notebooks covering API collection, web scraping, wrangling, EDA with visualization, EDA with SQL, the Folium map and the machine-learning prediction lab — all with their outputs saved.
- `spacex_dash_app.py`, the Plotly Dash application with the site dropdown, success pie chart, payload range slider and payload-versus-outcome scatter.
- The four Folium maps as standalone HTML files.
- Every figure used in this report.
- This PDF.
- Headline result: the Decision Tree classifier predicts the Falcon 9 first-stage landing outcome with 86.25% cross-validated accuracy and 83.33% test accuracy, which lets a competitor estimate the true cost of a SpaceX launch before it flies.